小项目与微弹药实施详解（白话版）

> 这份文档把《06-实施指南-小项目 S1–S5》里的每个项目和微弹药逐条展开，讲清楚每一步在做什么、怎么算过关。写法和两大项目的白话版一致，但小项目允许讲得朴素一些——**步骤一个都不跳**。原文档的定位铁律先抄在这里：小项目不写进简历，它们的用途只有两个——充当面试弹药，以及给大项目提前扫雷。评价标准只有一条：单位时间内能产出多少面试弹药。

这套小项目体系跟游戏里的支线任务差不多：主线（P1/P2）给你等级和装备，支线给你技能和弹药。每个小项目都是一个独立副本，进副本之前先想清楚要带走什么——出副本时手里必须攒齐”弹药三件套”（一个设计取舍、一个亲自踩过的坑、一张数据或对照表），攒不齐等于白打。主线 Boss 掉装备，支线副本掉技能，两样都拿到，秋招那场终局之战才打得动。

这份文档的用法：《06-实施指南》按项目分章，这份白话版沿用同样的顺序，逐个展开。每个项目的”步骤”是硬顺序——前面没过验收，后面不许开工，因为后面的实验全部建立在前面的代码上。每个项目末尾的”你的进度”告诉你这一关打完拿到了什么、对后面几关有什么用。

开工之前：所有小项目共用的五条纪律

1. 每个项目都要有独立仓库和 README（写清设计取舍、实验数据、复现命令）。代码是写给你自己看的，**真正的弹药是 README 和实验记录**；
2. 每个项目必须攒齐”弹药三件套”：**一个设计取舍、一个亲自踩过的坑、一张数据或对照表**；
3. **硬性时间盒**：开工之前在 README 顶部写下截止日期；超期了就砍功能，不能砍质量，更不许对自己说”再打磨一周”；
4. 项目做完立刻写一页《复盘卡》：如果重来会怎么改、被追问时最薄弱的三个点。面试之前你只需要看这张卡；
5. **名额纪律**：独立占用工期的小项目不超过 3 个——S1 必做；S6（平台岗）与 S4（存储岗）二选一；S3 等有余力再做；S2 并入 P1 不单列。微弹药 M1–M3 另算，总投入不超过 1 周。

这五条纪律的共同目的只有一个：**防止小项目膨胀成大项目**。小项目真正的敌人从来都是做得太多——每个超期的小项目都在偷 P1/P2 的工期。

名词小词典（按项目分组，用到的再看）

名词	大白话解释
S1 相关	
Raft	让多台机器对”操作日志的顺序”达成一致的算法，分布式系统的必修课。
任期（term）/ 三种角色	Raft 把时间切成任期；每个节点任一时刻是 leader（主）、follower（从）、candidate（候选人）之一。

名词	大白话解释
leader / follower	主节点接所有写请求；从节点只被动同步。leader 挂了大家重新投票选新的。
脑裂	极端情况下出现两个 leader，各写各的——一致性算法要防的头号事故。
WAL（预写日志）	先把操作追加写到磁盘日志、再应用到内存，崩溃后靠重放日志恢复。
crc32	一段数据的校验码，读回来对不上就说明数据坏了。
提交（commit）	一条日志被多数派节点复制成功，才算”确定下来了”。
线性一致读	读操作要能看到最新的写入。ReadIndex 和 Lease 是实现的两种办法。
S4 相关	
bitcask	一种存储引擎模型：数据只追加写日志文件，索引全放内存。
tombstone（墓碑）	删除操作的标记。bitcask 里删除也是一次写入，写一个特殊标记盖住旧值。
hint file	merge 时生成的”索引提示文件”，让重启后的扫描快得多。
merge（合并）	把多个旧文件里的存活记录整理进新文件，清掉垃圾数据。
LSM / SSTable / compaction	另一类存储引擎（LevelDB/RocksDB 一系）的关键词，S4 只做对读不做实现。
S2 相关	
io_uring	Linux 最新的高性能异步 I/O 接口，一次提交一批读写、一次收割一批结果。
队列深度（QD）	一次排在队列里的 I/O 数量。
O_DIRECT	绕过系统缓存直写磁盘，省一次拷贝，但对内存地址、长度、偏移有对齐要求。
overlayfs	容器镜像用的分层文件系统，在上面用 O_DIRECT 可能直接报错。
fio	磁盘性能测试的标准工具，用来测”裸盘上限”。
S3 相关	
eBPF	往内核里安全地挂小程序的机制，观测性能的神器。
tracepoint / kprobe	内核预留的固定观测点 / 可以挂在几乎任何内核函数上的探针。前者稳，后者灵活。
per-CPU map	每核一份数据的内核哈希表，避免多核抢锁。

名词	大白话解释
CO-RE	“一次编译到处运行”：让 eBPF 程序不重编译就能适配不同内核版本。
readahead（预读）	内核猜你要读后面的数据、提前搬进缓存的机制，它会干扰归因。
S6 相关	
scheduler-plugins	Kubernetes 官方的调度器插件仓库，S6 的基座。
扩展点（PreFilter/Score）	调度流程预留的挂钩位置。PreFilter 先筛掉不合适的节点；Score 给剩余节点打分。
NFD / NRT CRD	Node Feature Discovery 负责发现节点硬件拓扑并上报成 NodeResourceTopology 这个自定义资源。
Topology Manager	kubelet 里管 CPU/设备拓扑对齐的组件，v1.27 起 GA。
DRA	Dynamic Resource Allocation，K8s 新一代设备分配机制，v1.35 起 GA。
device plugin（旧模型）	老的设备上报机制，把设备抽象成整数（如 `nvidia.com/gpu: 1`），表达不了拓扑细节。
schedulerName	Pod 上指定”用哪个调度器调度我”的字段，自研调度器靠它接管流量。
微弹药相关	
namespace（命名空间）	Linux 的隔离机制：PID、网络、挂载点等各隔离一套。容器的地基。
cgroup	Linux 的资源限额机制：CPU、内存用量都能管。
pivot_root / chroot	切换进程根目录的两种手段，前者更彻底。
veth / bridge	一对虚拟网线 / 一台虚拟交换机，容器组网的最小积木。
netns	网络命名空间，一个独立的网络栈。
PV / PVC	持久卷 / 持久卷申领，K8s 里”存储资源”的供需两端。
bind mount	把一个目录原样挂到另一个路径，CSI 挂卷的最后一哆嗦。
CNI / CSI	容器网络接口 / 容器存储接口，两个标准化的插件协议。

S1 mini-Raft KV（必做，3–4 周硬顶）

做什么：用 Go 实现一个 3 节点的 Raft，对外提供 Get/Put 两个接口。目的：把一致性从”背出来的知识点”变成”能讲清楚的东西”。

先保持清醒：这是 MIT 6.824 里人手一个的项目，实现本身的差异化几乎为零，简历上永远不要把它排在第一位；它的弹药价值全在工程细节、实验设计和工业代码对读里。

范围：做选举、日志复制、快照、崩溃恢复；**不做** membership 变更和 Pre-Vote（这两个列为”必须能白板讲清楚”的内容）。

为什么 S1 必做、且放在最早：一致性是分布式系统面试的第一道分水岭，Raft 又是最适合亲手实现的一致性算法——资料全、边界清、能注入故障做实验。把它放在最早，因为它给后面所有项目托底：S6 的调度插件、S4 的存储引擎、P1 的缓存系统，面试官追问到最后都会落到一致性和故障语义上，而这些问题你在 S1 全部亲手碰过。

步骤 1：单机 KV 加 WAL

先不碰分布式，把单机版本做扎实：

1. 实现一个内存 KV 存储（`map[string]string` 加锁即可）；
2. 前面挡一层 WAL：每次 Put 先追加写日志、再改内存。日志记录格式 `len | crc32 | entry`，追加写；
3. **有三个字段必须持久化：`currentTerm`、`votedFor`、`log[]`**。这里埋着经典死亡陷阱：votedFor 不落盘，节点重启后就可能在同一个任期里投出两票，进而脑裂；
4. 启动时重放 WAL 恢复状态，用 crc 校验识别损坏的尾部记录。

   WAL 的两个工程细节：第一，追加写之后什么时候 fsync 是个权衡——每条都 fsync 最安全但最慢，攒一批再刷（group commit）快但崩溃时可能丢掉最后一批；S1 阶段先每条都 fsync 保正确性，性能权衡留给 S4 去实测。第二，重放时一旦某条记录 crc 校验失败，这条记录连同它之后的所有内容全部丢弃——损坏点之后的记录已经不可信，这个”尾部截断”逻辑是 WAL 的通用做法，etcd 和 PostgreSQL 都这么处理。

步骤 2：搭 gRPC 骨架和选举

1. 定义两个 gRPC 接口：`RequestVote` 和 `AppendEntries`（后面复制也用它）；
2. 三个角色一个 `tick` 驱动所有定时器；选举超时在 `[T, 2T)` 区间随机化，T 大约取 500ms——随机化是防止大家同时发起投票；
3. 实现完整的选举流程：follower 超时变 candidate → 给自己投票 → 向所有人拉票 → 多数票当选；
4. **收到每个 RPC 回复都先比较 term，发现自己的 term 更小就立刻让位变 follower**——这是第二大陷阱；
5. 日志里打印每次 term 变化和投票流向，调试全靠它。

   tick 驱动的具体形态：一个 goroutine 跑 `for { select }` 大循环，所有状态变更（角色切换、term 更新、投票记录）都只发生在这个循环里；RPC handler 只负责收消息、往循环的 channel 里塞事件。这个”单循环加 channel”的结构让你几乎不用加锁——Raft 的并发 bug 九成来自多个 goroutine 直接改共享状态，单循环从结构上避开这一整类 bug。etcd raft 库的 `node.go` 也是这个形态，步骤 5 对读时你会一眼认出来。

步骤 3：复制与提交

1. leader 收到写请求 → 追加到自己的日志 → 并行发 `AppendEntries` 给所有 follower → 多数派确认后提交 → 应用到状态机 → 回复客户端；
2. **提交规则：leader 只能提交当前任期的日志**（Raft 论文 §5.4.2 的结论，必须能讲为什么）；
3. 日志不一致时的修复：`prevLogIndex/term` 不匹配就回退 nextIndex 重发；如果你实现了冲突项的加速回退（一次跳过一整段冲突日志），这是加分项；

   加速回退的具体做法：朴素版是 follower 拒绝一次、leader 把 nextIndex 减 1 重发，一条一条往前试，冲突段长的时候要往返几十次。加速版让 follower 在拒绝回复里捎带两样东西——冲突日志条的 term 值、以及这个 term 在自己日志里的第一条索引；leader 收到后查自己的日志：自己也有这个 term，就把 nextIndex 跳到自己在该 term 的最后一条之后；没有，就直接跳到 follower 给的那条索引。这一跳把几十次往返压成一次，etcd 的 raft 库就是这么做的，你实现了它，对读时就能对上号。
4. 把 `raftd` 集成进测试 harness：能注入延迟、网络分区、进程重启，后面所有实验都跑在这个 harness 上。

步骤 4：快照和 kill -9 恢复

1. 日志太长时做快照：在 `lastIncludedIndex` 之前截断日志，快照保存状态机当前内容；
2. 落后的 follower 直接用 `InstallSnapshot` 传快照，不再逐条补日志；
3. 恢复顺序：先装快照，再重放快照之后的日志；
4. 测试方式简单粗暴：随时 `kill -9` 任意节点再拉起，验证数据不丢、集群能自愈。

   快照里到底存什么：状态机的全量数据（整个 KV map 的完整拷贝）加两个元数据——`lastIncludedIndex` 和 `lastIncludedTerm`。后者容易被漏掉：恢复快照之后，第一条新日志的 `prevLogIndex/term` 检查要靠 `lastIncludedTerm` 才能做，漏了它，快照之后的一致性检查就没着落。快照文件本身也要带 crc，写到一半崩溃的快照不允许被加载。

步骤 5：v3 的两组实验 + etcd 对读

1. **实验一（换血后的丢包扫描）**：丢包率取 0.1% / 1% / 5% / 10% 四档，画 commit 延迟 P50/P99 随丢包率变化的曲线，并解释拐点——重传与选举超时是怎么耦合的。老实验”30% 丢包单点”已删除，它太极端，只能得到”它瘫了”一个结论；
2. **实验二（新增，可用性数字）**：`kill -9 leader` 至少 100 次，记录从 kill 发生到新 leader 完成第一个成功写入的时间分布（中位数/P99）。分区实验区分三种：多数派隔离、少数派隔离、不对称分区（leader 单向失联），各给一张行为表；
3. **工业对读（必做）**：精读 etcd 的 raft 库——重点看 `node.go` 的 Ready 循环、`unstable` log、`Storage` 接口、ReadIndex 的工业实现。产出一篇《我的 toy raft 与 etcd raft 的 5 个差异》（候选差异：批量化的 Ready 通道、消息流水线、leader 转移、检查点时机、无环日志结构）；
4. 线性一致读是 stretch 目标：用 ReadIndex 或走日志实现，并能讲清楚”为什么 ReadIndex 对时钟的依赖比 Lease 小”。

丢包怎么注入：用 `tc qdisc add dev eth0 root netem loss 1%` 这类命令在网卡上注入丢包，harness 里把它包成”设置丢包率 → 跑负载 → 采集 → 清规则”的固定流程。注意 netem 的丢包是随机丢、不区分层次，你观察到的延迟劣化里有一部分是 TCP 重传贡献的、一部分是 Raft 重试贡献的——分析拐点时把这两层分开讲，才显得你真的看懂了曲线。

验收标准：故障注入下已提交数据零丢失；三类分区行为表完整；丢包扫描曲线画出来；无主窗口分布至少 100 个样本；etcd 差异笔记写完。

你能讲的点：votedFor 为什么必须持久化；为什么只能提交当前任期日志；ReadIndex 与 Lease 的对比；你的 toy 与 etcd 的具体差异；Pre-Vote 如何防止分区节点拿旧 term 扰乱选举；joint consensus 两阶段成员变更（不实现，但要能白板讲）。

> **你的进度**：S1 打完，分布式一致性这块的弹药就齐了——一套能跑故障注入的 3 节点 Raft、两组带曲线的实验数据、一篇 etcd 对读笔记。它还是后面所有项目的信任地基：面试官相信你能啃 K8s 调度插件（S6）和存储引擎（S4），很大程度上是因为你先证明了 Raft 这种硬骨头你啃得下来。

S2 C++/io_uring 预取器（并入 P1，不单列）

定位：P1 v3 已经把它收编为 A 轨（小文件高 IOPS 臂）的实验分支。可以单独建仓库，但它不占小项目名额、不另排工期，随 P1 步骤 8 前后推进。

范围：做本地写路径和提交模型；**不做**完整的 S3 客户端（HTTP Range 已够用）。

步骤 0：锁环境、练手感（v3 新增）

1. `uname -r` 锁定内核版本——registered files/buffers 的行为随版本变化；
2. **把 liburing 官方 examples 逐个通关**。io_uring、O_DIRECT、overlayfs 三个坑叠加，直接上手太陡，而 C++ 是你的次语言，这个坡必须先削平。

   为什么要先通关 examples：liburing 的 examples 每个只演示一个特性（fixed file、registered buffer、SQPOLL、linked SQE），每个几十行，半天能跑完。直接上手写预取器，你会把 io_uring 的坑、O_DIRECT 的坑、C++ 的坑混成一团，调试时根本分不清是哪一层出的问题。examples 通关相当于给每个特性单独校准一次手感。

步骤 1：Go 线程池加 buffered 基线

用 Go 写一个带线程池的 buffered I/O 写入器，跑出第一组基线数字。它是后面所有版本的比较原点。

步骤 2：C++ buffered 版本

同样的逻辑用 C++ 再写一遍。这一步的目的是**隔离语言因素**：之后 Go 版和 C++ 版的差异里，才能分清哪些来自 io_uring、哪些来自语言本身。

步骤 3：O_DIRECT 加 io_uring

1. 写路径换成 O_DIRECT 直写 + io_uring 异步提交；

2. **O_DIRECT 三重对齐的坑**：缓冲区地址、长度、偏移都必须对齐逻辑块，用 `posix_memalign` 对齐到 4K；
3. **overlayfs 的坑**：容器里的 overlayfs 上用 O_DIRECT 可能直接报 EINVAL——目标路径必须落在宿主机真实的 ext4/xfs 上（用 hostPath，或者干脆在宿主机上跑）；
4. 提交与收割：`get_sqe → prep → submit_and_wait(N)`，CQE 批量收割后再 resubmit，错误码逐个处理（短读即 EOF）。

   io_uring 的两种提交模式：默认模式每次 `io_uring_submit` 都有一次系统调用；开了 `IORING_SETUP_SQPOLL` 之后内核起一个线程帮你收割 SQ，用户态零系统调用——但 SQPOLL 线程空闲时会白白烧 CPU，小负载下反而更慢。两档都测一遍，把”什么负载下 SQPOLL 才划算”写进报告，这比单纯报一个峰值数字有说服力。

步骤 4：registered 优化版

用 `IORING_REGISTER_FILES` 和 registered buffer 消掉每个操作的固定开销；队列深度在 64–256 之间可配。

步骤 5：跑参数矩阵

队列深度 × 块大小（4K/64K/1M）× 并发扫点，记录吞吐、strace 统计的 syscall 数、CPU 占用。**用 fio 先测裸盘直写上限，贴着上限比**——“我比 fio 差 8%”这种说法比”我很快”专业十倍。

收益预期预先诚实化：P1 主臂的 4MiB 块经 HTTP 抓取时受网络限速，省 syscall 对吞吐的贡献预期接近零；io_uring 真正的战场是小文件高 IOPS。报告的正确写法：“syscall 数量大幅下降、大块吞吐不变（并解释为什么受网络限速）、小文件臂 IOPS 提升 X”，不许只挑好看的数字报。

弹药：O_DIRECT 对齐与 overlayfs 的坑；io_uring 到底减少了什么（syscall 加线程调度）；为什么大块场景下它赚不到收益——把”不适用的边界”讲清楚，比吹收益值钱得多。

> **你的进度**：S2 不占名额、不单列工期，但它产出的三条弹药——对齐坑、overlayfs 坑、收益边界——会直接并进 P1 的 A 轨报告。这一关最大的收获是一句认知：io_uring 有明确的适用边界，讲清边界比吹嘘收益专业得多。

S3 eBPF 观测工具（可选，P1 核心 1 完工之后再做，1–2 周）

定位：原来排在 P1 指标期会跟核心 1 争时间，现在后移；MVP 砍到只剩一个探针。范围只做只读观测；内核版本不低于 5.10；在宿主机上跑（容器权限很麻烦）。

什么时候真的去做它：只有一种情况——P1 核心 1 已经完工、距下一个里程碑还有一两周空档。其他任何时候它都排在 P1/P2 后面，不许插队。

步骤 1：bpf2go 的 hello-world

用 bpf2go 编译并加载一个最小 eBPF 程序，验证环境（clang、内核头、vmlinux.h、CO-RE）全部就绪。跑不通环境，后面免谈。

步骤 2：block I/O 延迟直方图（唯一必做项）

1. 挂两个 tracepoint：`block/block_rq_issue`（下发）和 `block/block_rq_complete`（完成）；
2. 按 request 指针把两次事件配对，算出每个 I/O 的延迟；
3. 在内核侧按 log2 bucket 聚合（别把每个样本都送到用户态）；用户态周期性拉取 per-CPU map；
4. 用 Go 把直方图画出来或导出成指标。

   配对的工程细节：request 指针作为 map 的 key，issue 事件存时间戳，complete 事件查到就做差。要防两件事——map 只增不删会撑爆（issue 了但永远没 complete 的请求要定期清理）；request 指针被内核复用会错配（清理窗口太长时，老条目和新请求撞 key）。这两件事处理不好，直方图里就会混进莫名其妙的超长延迟样本。

步骤 3（可选）：再挂两个探针

major fault 探针（`exceptions/page_fault_user` 里的 major/minor 位）和 `sys_enter_read` 字节量探针。有余力再做。

步骤 4：接入 P1 实验，交叉验证

1. 把 exporter 接入 P1 的实验流程，与 dataloader 的打点交叉验证口径；
2. **有一条观测逻辑必须写进报告**：P1 的 preheat 模型下，缓存命中 page cache 时根本不会产生 block I/O——所以 block I/O 直方图天然就能区分命中与未命中；
3. **readahead 陷阱单独成节**：内核预读造成的 major fault 不会记在消费者进程名下，按进程维度归因就会算错。这个陷阱本身就是上等弹药；
4. 准备降级方案：eBPF 挂不上时用采样 `/proc/diskstats` 兜底，并诚实标注两个口径的差异。

验收标准：P1 实验期间能稳定出数；两个口径交叉验证一致；readahead 归因陷阱有专门一节。

> **你的进度**：S3 是可选关，打完它，P1 实验的观测维度就从用户态打点升级到内核态直方图。readahead 归因陷阱这条弹药在任何存储或性能面试里都拿得出手。

S4 bitcask mini 存储引擎（存储岗首选，与 S6 二选一，3–4 周）

做什么：用 Go 实现一个 bitcask：WAL 加内存索引，加 hint file，加 merge。

范围：只做单机；**不做**分布式（S1 已覆盖）、完整的 LSM（只做对读）。

为什么存储岗首选它：存储引擎的面试主线就三条——写路径（WAL/追加写）、读路径（索引/缓存）、空间回收（merge/compaction），bitcask 三条全占，而且每条都足够小、能手写。做完它再对读 LSM，你就同时摸到了两大存储引擎家族的骨架。

步骤 1：append-only log 加读路径

1. 记录格式：`crc32 | tstamp | keysz(4) | valsz(4) | key | value`。**时间戳用 8 字节**——4 字节秒级时间戳有 2038 问题，记录头统一用 int64 毫秒/微秒；主动在注释里写出这个修正，小细节但能留下大印

象；

2. 删除操作也是一次写：tombstone 用 valsz 的特殊值标记；
3. 内存索引：`map[key] → {fileID, offset, size}`；
4. 读路径：索引命中后用 pread 读取，再做 crc 校验——校验失败意味着盘上数据腐坏，这时报错或触发重新 merge。

   读路径的完整链路：`Get(key)` 先查内存索引拿到 `{fileID, offset, size}`，再对那个数据文件做一次 `pread`（带偏移量的系统调用，不用 seek，天然线程安全），读回完整记录后先验 crc、再解出 value 返回。整条路径只有一次磁盘 I/O——这是 bitcask 最大的卖点，也是它必须把索引全放内存的原因：索引一旦放不进内存、需要换入换出，“一次 I/O”的保证立刻破功。

步骤 2：多文件轮转加 tombstone

1. 活跃文件写满 256MB 后转为 immutable，新开一个活跃文件接着写；
2. 删除写入 tombstone，读到墓碑就报”不存在”；
3. **fsync 三档做成配置**：逐条刷盘 / group commit（N 毫秒）/ 只靠操作系统。三档拿来 benchmark 对比，这是存储引擎最核心的权衡之一；
4. 明确实现**单写多读**模型：写锁天然串行化（这是 bitcask 和 LSM 共同的核心约束），读操作无锁；后面压测要验证多写者的排队情况和多读的扩展性。

步骤 3：启动扫描

1. 第一版暴力扫：启动时把所有数据文件从头到尾读一遍重建索引；
2. 第二版用 hint file：merge 时顺手生成的索引提示文件，启动只读它；
3. 两种方式的启动时间做成对比数据。

   hint file 的格式：每条记录只存 `tstamp | keysz | valsz | offset | key`——比数据文件里的完整记录少了 value，所以个头小得多。启动时顺序读一遍 hint 文件就能重建整个索引，不用再扫全量数据。hint file 是 merge 的副产品：merge 本来就要遍历所有存活记录，顺手把每条记录的索引信息写一份，几乎不花额外成本。

步骤 4：merge 加 manifest 原子切换

1. merge：把 immutable 文件中的存活记录写入新数据文件和 hint file；
2. **原子切换的顺序**：先 rename 数据文件和 hint 文件，最后写 manifest 指向新的文件集合；
3. 把每一个崩溃窗口想清楚、写进注释：rename 到一半崩溃会怎样？manifest 写一半呢？旧文件什么时候才能删？

   三个崩溃窗口的标准答案：rename 序列做了一半崩溃——单个 rename 是原子的，但多个文件的 rename 序列可能只做了一半；重启后以 manifest 为准：manifest 没指向的新文件就是孤儿，直接清理。manifest 写一半崩溃——manifest 必须先写临时文件、再 rename 替换，rename 的原子性保证读到的一定是完整的旧版本或新版本，不存在”半个 manifest”。旧文件删之前崩溃——没关系，重启时发现 manifest 没引用它们，当垃圾回收。这套”先写临时文件、rename 切换、manifest 收口”的三板斧，是所有存储引擎做原子切换的通用模式，LevelDB 的 MANIFEST 机制长得几乎一样。

步骤 5：fuzz 崩溃矩阵

用 fuzz 随机注入 kill -9（在写、在 merge、在 rename 等各种时机杀），跑完崩溃矩阵，验证任何时刻重启都能恢复到一致状态。

fuzz 矩阵的具体形态：两个维度正交——崩溃时机（写 WAL 中途、写 tombstone 中途、merge 写新文件中途、rename 之间、manifest 写入中途）× 崩溃方式（kill -9、用 `/proc/sysrq-trigger` 模拟断电）。每一格跑若干次，每次重启后做全量校验：所有 key 要么读到最后一次写入的值、要么读到墓碑。矩阵全绿之前不许进步骤 6。

步骤 6：benchmark

1. **1M/10M 两档都要跑**（32G 内存的机器跑得动 10M 档）：1M 档算不出 bitcask 的命门——全内存索引的容量账。实测启动时间、常驻内存、读写吞吐，画出”索引内存/数据量”的比值曲线；
2. merge 期间的写延迟抖动——**再丑的数字也要敢贴**；
3. 写放大计算（写入磁盘的字节数 ÷ 有效数据量）；
4. fsync 三档对比曲线；
5. 多写者排队与多读扩展性压测。

   容量账怎么算：10M 键值、平均 key 16 字节时，内存索引每条记录约 40 字节（key 加 fileID、offset、size，再算 map 开销），总索引约 400MB——游戏本扛得住，但数据再涨 10 倍就悬了。把这个推算过程写出来，就是”bitcask 什么时候输给 LSM”的定量答案：bitcask 的索引内存随 key 数量线性涨，LSM 的稀疏索引只随数据文件数涨。

步骤 7：LSM 对读（不实现）

读 LevelDB/RocksDB 的 compaction 文档，最终目标是在白板上回答：“数据量超过内存之后，bitcask 为什么会输给 LSM？”（对比 SSTable、稀疏索引、分层 compaction 的读写放大）。

验收标准：崩溃矩阵全绿；10M 档容量账有实测；merge 抖动数据完整；LSM 对读笔记一页。

弹药：fsync 三档；merge 原子切换的崩溃窗口；单写多读约束；10M 实测内存账；bitcask 与 LSM 的适用边界。

> **你的进度**：S4 打完，存储岗的面试弹药就齐了——从 WAL 讲到 merge、从 fsync 权衡讲到写放大、从 bitcask 讲到 LSM 边界。它和 S1 合起来覆盖了单机存储和分布式一致性两个域，投存储岗时这两关的弹药比 P1 本体还直接。

S6 K8s 调度插件实战（平台/AI Infra 岗首选，与 S4 二选一，3–4 周）

做什么：基于 kubernetes-sigs/scheduler-plugins，先跑通官方的拓扑感知调度，再自研一个最小的 PreFilter+Score 插件走通全链路，最后在”数据亲和 × 拓扑亲和”的冲突点上做增量实验。

为什么选它：它与 P1 咬合最紧——P1 核心 1 就是 Gate 加 Score plugin，这个项目相当于提前给 P1 步骤 7 扫雷；NUMA 和设备拓扑是平台岗高频题；scheduler-plugins 是官方仓库，还能顺带练习开源流程（但它不

背开源名额，主要回馈目标仍然是 Fluid/AIBrix）。

开工第一件事：硬件诚实清单

1. 先跑一次 `numactl -H` 确认：游戏本的消费级 CPU 几乎肯定是单 NUMA 节点，也没有多 GPU；
2. 所以所有拓扑都用 fake/annotation 驱动来开发——**这完全正当**，官方插件自己的 e2e 测试也在 mock NRT 对象；
3. 真机多 GPU 拓扑只在 P2 租云主机的那个窗口顺带读一次（`nvidia-smi topo -m`），不值得单独开预算。

   fake 拓扑的正当性要敢讲：面试官如果质疑”你的拓扑全是假的”，标准回答是——调度插件的输入是 NRT 对象，NRT 对象的内容来自哪里（NFD 上报还是手写 yaml）对插件逻辑完全透明；官方 e2e 测试同样手写 NRT。你测的是”插件对拓扑信息的消费逻辑”，这个逻辑在真假拓扑面前一视同仁。

DRA 白板要点（不写代码，与阶段 A 同期写半页笔记，四条必须脱稿讲）

1. **旧 device plugin 模型的痛点**：设备被抽象成整数资源（如 `nvidia.com/gpu: 1`），没法按显存、型号或互联拓扑发结构化请求，不支持设备共享，拓扑信息只能走 Topology Manager + NFD + NRT 这条外挂链路；
2. **DRA 的四个核心对象**：驱动用 **ResourceSlice** 上报设备能力与拓扑；管理员定义 **DeviceClass**（类比 StorageClass）；工作负载用 **ResourceClaim / ResourceClaimTemplate** 申领设备（体验类似 PVC/PV）；调度器做结构化匹配，kubelet 与驱动完成分配握手。支持 CEL 属性过滤和设备共享；
3. **它与 NRT 的关系**：DRA 让设备拓扑成为调度器可直接消费的结构化属性，长期会收编 NFD+NRT 链路的一部分；但迁移期两套并存，NRT 插件在现网仍被广泛部署。能讲清”哪部分会被取代、哪部分（NUMA/CPU 亲和）仍会独立存在、迁移卡在哪（驱动成熟度与存量集群版本）“才是满分答案；
4. **现状判断（2026-10 核实）**：DRA 从 v1.35 起 GA（stable），v1.36 把 ResourceClaim 接入了 Workload/PodGroup 体系。**禁止再说”DRA 是未来的实验特性”**；v1.36 的 gang scheduling 与 P1 的训练 Job 调度是同一个演进方向，当谈资，不跟进实现。

可选的两小时手感练习（不过夜、不写简历、不挤占 P1）

阶段 A 完成之后，用 kind 建一个 1.35 以上的集群，跑官方的 `dra-example-driver`（不需要真实硬件的假设备驱动），创建一个 DeviceClass 加一个 ResourceClaim，`kubectl describe resourceclaim` 能看到分配结果和 ResourceSlice 内容就收工。目标只是”见过真的 ResourceSlice 长什么样”。

阶段 A：现勘并跑通官方链路（约 1 周）

1. 搭好 kind 集群（版本对齐 scheduler-plugins 的兼容矩阵），按仓库 Developer Guide 在本地构建镜像（kind load）或直接用发布的 chart；
2. 部署 NFD 加 topology-updater，生成真实的 `NodeResourceTopology` CR；在单 NUMA 机器上手写一份 fake NRT yaml（构造双 NUMA、fake GPU 资源池），作为第二个数据源；
3. 配置 **v1 版本**的 `KubeSchedulerConfiguration`：`profiles`（schedulerName）、`plugins`（在 filter/score 上启用 NRT 插件）、`pluginConfig`（scoringStrategy：MostAllocated、BalancedAllocation、LeastAllocated 各跑一遍，观察决策差异）；
4. 部署若干请求 CPU/GPU 的测试 Pod，验证绑定与拓扑决策符合预期，抓取调度日志；

三种 scoringStrategy 要看什么：MostAllocated 偏好资源用得多的节点（堆叠，省碎片），LeastAllocated 偏好空闲节点（打散，抗突发），BalancedAllocation 折中。同一批测试 Pod 在三种策略下的落点分布画成一张表，这张表就是”调度策略影响落地”的最直观证据，面试时直接拿出来讲。

5. 精读官方 Score 插件源码（分数如何由 NRT 的 zones 和资源分配算出、如何归一化）和 NRT CRD 的字段语义；
6. **验收（能口述全链路）**：device plugin 的 `TopologyInfo` → kubelet Topology Manager → NFD topology-updater → NRT CRD → 调度插件 Filter/Score → bind；
7. **风险保护**：一周内官方全家桶部署不成功（NFD 组件多、镜像坑多），S6 立即降级为”手写 NRT yaml + 只部署插件本体”；再不行降级为”纯跟读源码 + 写决策日志推演笔记”，不能让环境黑洞吃掉第二周。

阶段 B：自研最小插件（1–2 周）

1. 插件骨架：实现 `New(obj runtime.Object, handle framework.Handle) (framework.Plugin, error)`，挂上 **PreFilter 与 Score 两个扩展点**（NormalizeScore 视需要再加）；
2. 配置：在 `New` 里读取 pluginConfig 参数（权重、annotation key），存进插件状态；
3. **拓扑数据源刻意简化**：先读 node annotation（如 `topology.io/numa-zones`，里面写 fake 拓扑 JSON），不依赖 NFD 全家桶——这一步的目标是走通链路；
4. 打分逻辑：根据 Pod 请求的资源与节点的 zone 拓扑，计算”单个 NUMA 内的可满足度”分数；PreFilter 先排除明显不可行的节点；

   NormalizeScore 什么时候必须加：框架会把各插件的 Score 按权重合成，但它不管你的原始分数值域——你的 Score 输出 0–100，别人的输出 0–1，加权合成就全乱了。NormalizeScore 的作用是在所有节点打分完之后，把你的分数统一压到框架约定的 [0,100] 区间。单插件场景可以不加，多插件并存（比如阶段 C 和 NRT 插件一起跑）时必须加。

5. 工程配套做全：自定义 scheduler 的 Deployment、ServiceAccount/RBAC（这份权限清单本身就是弹药）、ConfigMap 挂载 KubeSchedulerConfiguration；
6. Profile 配置：用 YAML 定义 profile 与插件启用顺序；本地调试按 Developer Guide 来（本地跑 scheduler 进程连 kind apiserver，或容器化调试，二选一）；
7. 质量：插件单测（构造 nodes/pods 的 table tests）+ e2e（调度结果符合 annotation 预期）+ 每个调度决策一条结构化日志；
8. **验收**：在一个独立的 schedulerName 上，“配置 → 编译 → 部署 → 调度生效”全链路走通，单测和 e2e 全绿。

阶段 C：双目标冲突实验（1–2 周，与 P1 步骤 7 合流，不额外占工期）

1. 让两个 Score 维度并存：**数据亲和分**（P1 的 Job 子集 readyRatio，由 P1 的 informer 缓存提供）× **拓扑亲和分**（fake NUMA/GPU zone 匹配度）。**合成公式先写死再实验**：`score = w·数据亲和分 + (1−w)·拓扑亲和分`，两个分量都必须归一化到 [0,1] 再合成——数据亲和分 = readyRatio（天然在 [0,1]），拓扑亲和分 = 单个 NUMA 内可满足的资源种类数 ÷ 请求的资源种类总数（也在 [0,1]）。不归一化的话，两个分量的值域不一致，w 的权重就没有意义，扫出来的曲线也没法解释——这和 P2 路由打分的归一化是同一个道理；

2. 构造冲突场景：Job 的热分片缓存在 NUMA-A 节点上，但空闲 GPU 在拓扑上更远的节点；或者另一个节点 NUMA 局部性很好但数据是冷的；
3. 权重 w 从 0 扫到 1，画出**权重-调度决策曲线**，以及对应的 makespan/拓扑惩罚代理指标；
4. 把终点问题写成报告：框架级多插件的分数如何归一化（各插件 Score 的值域不同）、冲突时为什么不存在银弹、生产环境应该让谁把权重暴露给用户；
5. **合流动作（哪些资产搬进 P1、哪些只留在 demo）**：直接搬进 P1 步骤 7 的有四件——PreFilter/Score 插件骨架（`New` 函数、扩展点注册、框架接口的写法）、pluginConfig 的配置读取代码、RBAC 权限清单（ServiceAccount/ClusterRole/Binding 那套 yaml）、测试 harness（table tests 加 e2e 的脚手架）；只留在 S6 demo 版的有两件——fake 拓扑数据源（node annotation 那套，P1 会换成真实的缓存 informer）和拓扑亲和分的计算逻辑（P1 的打分维度不同）。“双分量加权合成”这个模式两边通用，但具体分量各自实现。这样划清楚是为了防止一种浪费：demo 版里精心打磨的 fake 拓扑逻辑如果原样搬进 P1，就成了没人用的死代码；
6. **验收**：产出一张冲突权重-决策曲线图；DRA 四条白板要点能脱稿讲。

弹药三件套：取舍（NRT annotation fake 与 NFD 全家桶之间怎么选、双目标如何归一化）、坑（KubeSchedulerConfiguration 的 apiVersion、多 profile、schedulerName、RBAC 权限、kind 镜像加载）、数据（权重-决策曲线加 scoringStrategy 对照）。

诚实清单：拓扑全部是 fake 的；官方 NRT 插件是基线，并不是要打倒的对手；没有做绑核验证，没有真机性能数据；DRA 没有动手实现，但它的现状（v1.35 已 GA）和四个要点必须跟到能讲、能答迁移期共存问题的程度。

> **你的进度**：S6 三个阶段打完，P1 步骤 7 的调度插件等于提前排练了一遍——骨架、配置读取、RBAC、测试脚手架全套现成，P1 只需要换掉打分逻辑。这一关还附赠 DRA 的四条白板要点；2026 年的平台岗面试里 DRA 是高频新题，能脱稿讲清楚的候选人不多。

微弹药 M1–M3（按天计的练手小品，算不上项目）

定位纪律（违反了它们就失去意义）：

1. **它们不算项目**：一个目录加一份 `NOTES.md` 就够，不做 README 包装、不写进简历、不占小项目名额。“我写过、代码就在这里、原理我讲得清楚”就是真弹药；
2. **硬性时间盒**：单个不超过 2–3 天，三个全做的净工作量不超过 1 周；一旦超时，立即走该条目的降级路径，不过夜、不打磨；
3. **窗口纪律**：只允许在寒假、实习空档或两个大项目的里程碑之间做；P1 核心 1 冲刺期（步骤 7 前后）禁止开工；
4. **顺序**：M1 所有人必做；M2/M3 按岗位选——走平台/调度方向优先 M2，走存储/数据流方向（P1 方向）优先 M3。

M1 mini-container（必做，1–2 天，性价比最高）

做什么：用 Go（或直接用 C 调 clone）写一个 100–200 行的容器运行器。

1. 用 `clone` 加 `CLONE_NEWPID / NEWUTS / NEWNS` 三个标志创建子进程（有余力再加 NEWNET/NEWUSER）——它就有了独立的进程号空间、主机名和挂载表；
2. cgroup 用 **v2**：在 `cgroup.controllers` 体系下写 `cpu.max`、`memory.max`、`memory.swap.max`，把子进程的 pid 写入 `cgroup.procs`，资源限额就生效了；
3. rootfs 用 busybox 或 distroless 的根目录，调用 `pivot_root` 切换（入门版可以先用 chroot），再挂载 `/proc`、`/tmp`；
4. stretch（可选）：用 overlayfs 的 lower/upper/merged 做一个可写层——正好与 S2 里”overlayfs 上 O_DIRECT 的坑”呼应。

验收标准：容器内执行 `ps` 只能看到自己的进程树；hostname 相互独立；把 `memory.max` 设成 64M 后跑内存分配，能触发 cgroup OOM 并观察宿主行为；用 `lsns` 能看到新的命名空间。

你必须能讲：容器为什么不是虚拟机（共享宿主内核意味着什么、攻击面在哪里）；cgroup v2 相对 v1 的统一层级变化；pivot_root 与 chroot 的区别；为什么 user namespace 是 rootless 容器的关键；containerd/runC 在 K8s 链路中的位置（kubelet → CRI → containerd → runC）。

降级路径：pivot_root 卡住就退回 chroot 加 mount namespace，核心目标（找到 namespace/cgroup 的手感）不打折。

实现顺序提醒：先 namespace、后 rootfs、再 cgroup—— `clone` 创建隔离环境，`pivot_root` 切换根目录，最后写 cgroup 限额。挂载 `/proc` 必须放在 pivot_root 之后，顺序反了，容器里的 `ps` 看到的就是宿主的进程表，CLONE_NEWPID 白做。这类”顺序错了全盘皆输”的细节在容器运行时里到处都是，亲手踩一次比读十篇文章记得牢。

M2 mini-CNI（平台/网络向，1–2 天）

做什么：写一个能被 CNI 规范真正调用起来的最简网络插件（Go 可执行文件）。

1. 实现 CNI 规范的 `ADD` / `DEL` / `CHECK` 三个命令：从 stdin 读 NetConf JSON，从 `CNI_ARGS` 和环境变量里拿 netns 路径；
2. bridge 模式：建好 bridge，给容器 netns 创建 veth pair 并把一端塞进去，从 /24 地址池分配 IP（用位图或租约文件持久化，防止 ADD 重启之后地址泄漏），配好接口地址与默认路由，再把 lo 拉起来；
3. 用 `cnitool` 在两个独立的 netns 上验证互通；DEL 时释放 IP、删掉 veth。

验收标准：两个 netns 之间能 ping 通；重复 ADD 不会撞 IP；DEL 之后地址正确回收；用 `tcpdump` 能抓到 bridge 转发的包。

你必须能讲：K8s 的”IP-per-pod”网络模型；CNI 在容器创建链路里哪个时刻被调用（netns 已建好、rootfs 已就位之后，由运行时/CRI 调用）；跨节点流量怎么办（VXLAN、路由、BGP——这个练习故意不做，但你要能说清边界在哪里）；service 的 iptables/IPVS 与 CNI 为什么是两层东西。

降级路径：CNI 调试吃掉半天还没结果就停下，改为读 containernetworking/plugins 的 bridge 插件源码，再用 `ip link/addr/route` 手敲复现全链路，然后写笔记。

一个值得记录的坑：veth pair 的一端塞进容器 netns 之后，要在容器 netns 内部把接口 up 起来并配地址——在宿主侧配的地址不会跟着接口走。IP 分配的租约文件要加文件锁，两个 ADD 并发到来时没锁就会发出同一个 IP。

M3 mini-CSI（存储/数据流向，2–3 天，与 P1 咬合最紧）

做什么：写一个最小的 CSI 驱动，让它真的被 kubelet 调起来，让 Pod 能挂载你提供的卷。

1. 实现三个 gRPC service：`Identity`（GetPluginInfo/Capabilities/Probe）、`Controller`（最小实现，先不做 provisioning）、`Node`（在 `NodePublishVolume` 里把宿主目录 bind mount 到 kubelet 给的 target path；`NodeUnpublishVolume` 反向卸载）；
2. 走 **kubelet 的 plugin registration 机制**：在 `/var/lib/kubelet/plugins_registry/` 暴露注册 socket，与 kubelet 完成握手——这一步本身就是最有价值的弹药；
3. 先打通 **static PV 加你的 CSI driver** 的绑定挂载闭环；stretch 目标：实现 `CreateVolume/DeleteVolume`（在宿主上建/删目录）加 StorageClass 动态供给；
4. 部署形态二选一，先跑通简单的那种：kind 里用 DaemonSet（privileged、挂 hostPath），或者直接作为节点进程跑。

验收标准：PVC 完成绑定 → 测试 Pod 挂载成功并能读写文件 → 删掉 Pod/PVC 之后 bind mount 被干净卸载（用 `findmnt` 验证没有残留）。

你必须能讲：CSI 三段式流程——Attach（ControllerPublish）、Stage（NodeStageVolume，块设备格式化并挂到全局挂载点）、Publish（bind mount 到 Pod）各自解决什么问题，为什么最简实现只需要 Publish；kubelet-registration 握手与 socket 的生命周期；ephemeral 卷与 persistent 卷的区别；**Fluid/JuiceFS 为什么需要自己的 CSI driver**（FUSE 挂载如何经由 CSI 注入 Pod）——最后这一问直接和 P1 现勘连起来。

降级路径：动态供给做不完就停在 static PV 闭环（八成弹药已到手）；在 kind 里注册不上，就在节点上用 `csc` 工具直接调你的 gRPC 方法来验证。

注册握手的关键点：plugin registration 是双向的——你的驱动在 plugins_registry 目录暴露一个 socket，kubelet 的 watcher 发现新 socket 后主动来调你的 `GetPluginInfo`，拿到你的 CSI socket 地址之后再去调 Node service。kubelet 重启或你的驱动重启后握手要重做，socket 文件的生命周期管理（驱动退出时删掉自己的 socket）是这里最容易出错的地方。

微弹药与面试域的关系

微弹药	覆盖的面试域	联动
M1	容器运行时、namespace/cgroup、CRI 链路	S6（K8s 的下层）、S2 的 overlayfs 坑
M2	CNI、容器网络模型、netns	S6 调度之后”Pod 如何落地并联网”
M3	CSI、kubelet 卷生命周期、存储编排	**P1 现勘（Fluid/JuiceFS CSI）**、S4 存储岗叙事

完成顺序与依赖

```
现在（使用阶段）
  ├─ S1 起步（穿插进行，3–4 周硬顶；在 P1 早期完成）
  └─ P1/P2 双现勘（主战场）
P1 主线期
  ├─ S6 阶段 A/B（用周末预算，约 2–3 周）
  ├─ P1 步骤 7 ← S6 阶段 C 合流（不另占工期）
  └─ S2 随 P1 A 轨推进（不单列）
实习投递（2027.2–4）：P1 核心 1 + S1 + S6（平台岗）/ S4（存储岗）+ Fluid issue
P1 里程碑之间 / 寒假：M1 必做（1–2 天）；M3 优先（存储/P1 向，2–3 天），M2 面向平台岗
暑期实习后 / 秋招前：S3 用来填空档（前提是 P1 核心 1 已经完工）；如果改投存储岗，S4 放在这个窗口做
```

选岗规则：主攻 AI Infra 平台/调度方向，选 S6 加 M2；主攻分布式存储/数据流方向，选 S4 加 M3。不要贪心，正式小项目二选一；M1 无条件做，M2/M3 按岗位选，微弹药总耗时全部加起来不超过 1 周。

> **你的进度（全部小项目收尾时）**：到这里你的弹药库应该长这样——S1 给你一致性，S6 或 S4 给你岗位方向的纵深，S2 给你 I/O 边界认知，M1–M3 给你容器、网络、存储三条链路的底层手感。每件弹药都有”取舍、坑、数据”三件套撑着。面试前不用翻代码，把每个仓库的《复盘卡》过一遍就够——小项目存在的全部意义就在这张卡上。